

MACHINE LEARNING

LAB MANUAL 02

Course	Machine Learning
Lab Title	Machine Learning as Function Approximation OOP Implementation
Instructor	Muhammad Khalid Khan
Semester	Fall 2026

Majority Vote • Memorizer • Decision Stump • Decision Trees

Learning Objectives

- Review classes, objects, constructors, methods, attributes, and inheritance in Python.
- Understand supervised learning as approximation of an unknown target function.
- Explain input space X , output space Y , target function c^* , hypothesis space H , and hypothesis function h .
- Implement Majority Vote, Memorizer, and Decision Stump using the OOP paradigm.
- Evaluate classifiers using training/test error and 0–1 loss.
- Explain the relationship between a Decision Stump and a Decision Tree.
- Identify the limitations of simple baseline algorithms and the benefits of classical ML algorithms.

1. Lab Overview

This laboratory introduces supervised machine learning as a problem of function approximation. Students first recap object-oriented programming (OOP) in Python and then implement three basic classification algorithms using classes and objects: Majority Vote, Memorizer, and Decision Stump. The lab then connects the Decision Stump to Decision Trees and discusses why classical machine-learning methods provide stronger learning, generalization, evaluation, and modeling capabilities than these basic techniques.

Note: The conceptual sequence follows the supplied lecture slides: supervised learning → function approximation → hypothesis space/function → evaluation → basic classification algorithms → Decision Tree preview. The slides define the target function c^* and candidate hypothesis functions h , and distinguish classification from regression.

3. Recap: Object-Oriented Programming in Python

Object-oriented programming organizes a program around classes and objects. A class defines the data and behavior of an object. In this lab, each learning algorithm will be represented as a class with methods such as `fit()` for learning from training data and `predict()` for producing a prediction.

3.1 Core OOP Concepts

- Class: blueprint for creating objects.
- Object: an instance of a class.
- `__init__()`: constructor used to initialize object attributes.
- Attributes: data stored inside an object.
- Methods: functions defined inside a class.
- Encapsulation: keeping an algorithm's learned state and operations together.
- Inheritance: allowing a specialized class to reuse a parent class.

3.2 Small OOP Example

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def result(self):
        if self.marks >= 50:
            return "Pass"
        return "Fail"

student = Student("Ali", 72)
print(student.name)
print(student.result())
```

Expected output:

```
Ali
Pass
```

4. Function Approximation

In the function-approximation example, the input x belongs to an input set X , the output y belongs to Y , and an unknown target function $c^*: X \rightarrow Y$ generates the true outputs. The learner receives training examples and produces a hypothesis $h(x)$ that attempts to approximate $c^*(x)$.

For classification, outputs are discrete. For regression, outputs are real-valued. The lecture's example uses a target function such as $c^*(x) = \sin(x)$ and asks how well a learned hypothesis can approximate it.

4.1 Key Terms

Input space (X): The set of all possible inputs or feature vectors.

Output space (Y): The set of possible outputs or labels.

Target function (c*): The unknown true mapping from X to Y .

Training dataset (Dtrain): Examples $(x(i), y(i))$ used by the learner.

Hypothesis (h): A candidate function used to approximate the target function.

Hypothesis space (H): The set of candidate hypothesis functions available to the learner.

Prediction: The output $h(x)$ produced by the learned hypothesis.

4.2 Medical Diagnosis Example

The lecture models medical diagnosis as supervised binary classification. Each patient is represented by features such as hives, sneezing, red eye, and whether the patient has a cat. The label indicates the diagnosis.

```
# Example representation
X = [
    ["Y", "N", "N", "N"],
    ["N", "Y", "N", "N"],
    ["Y", "Y", "N", "N"],
    ["Y", "N", "Y", "Y"],
    ["N", "Y", "Y", "N"]
]

y = ["-", "-", "+", "-", "+"]
```

A simple hypothesis from the lecture is: predict '+' if sneezing is Y; otherwise predict '-'. This is an example of a hypothesis function based on one feature.

5. Hypothesis Space

The hypothesis space H is the collection of candidate functions from which the learner can choose. A learner cannot consider every imaginable function, so the selected hypothesis space imposes an inductive bias. For example, a Decision Stump restricts the learner to hypotheses that make a prediction using only one feature.

```
# Conceptual view
H = {
    h1, h2, h3, ..., hk
}
```

```
# A learner chooses one hypothesis:
h = LearningAlgorithm(Dtrain)
```

6. Evaluating a Hypothesis

The slides define a loss function as a measure of how bad a prediction is compared with the true output. For classification, binary/0-1 loss is 1 when the prediction is incorrect and 0 otherwise. For regression, squared loss is $(y - \hat{y})^2$. The lecture emphasizes evaluating on a separate test dataset to estimate performance on unseen data.

```
def classification_error(y_true, y_pred):
    mistakes = 0

    for actual, predicted in zip(y_true, y_pred):
        if actual != predicted:
            mistakes += 1

    return mistakes / len(y_true)
```

Example:

```
y_true = ["+", "-", "+", "+"]
y_pred = ["+", "+", "+", "-"]

print(classification_error(y_true, y_pred))
# 0.5
```

7. Algorithm 1 — Majority Vote

The lecture defines Majority Vote as: predict the most common label in the training dataset. The algorithm ignores the input features and uses only the distribution of labels.

7.1 How It Works

1. Count how many times each class appears in the training labels.
2. Select the most frequent class.
3. Return that class for every future input.

7.2 OOP Implementation

```
from collections import Counter

class MajorityVoteClassifier:
    def __init__(self):
        self.majority_label = None

    def fit(self, X, y):
        counts = Counter(y)
        self.majority_label = counts.most_common(1)[0][0]
        return self

    def predict(self, X):
        return [self.majority_label for _ in X]
```

```
X_train = [
```

```

["Y", "N"],
["N", "Y"],
["Y", "Y"],
["Y", "N"],
["N", "Y"]
]
y_train = ["-", "-", "+", "-", "+"]

model = MajorityVoteClassifier()
model.fit(X_train, y_train)

X_test = [["Y", "Y"], ["N", "N"]]
print(model.predict(X_test))

```

Possible output:

```
['-', '-']
```

Note: If '-' is the most frequent training label, every test example receives '-'. The algorithm does not inspect the feature values.

7.3 Limitations

- Ignores all features, so it cannot learn relationships between inputs and outputs.
- Produces the same prediction for every input.
- Can perform poorly when classes depend strongly on feature values.
- A class-imbalanced dataset can make the algorithm appear accurate while ignoring minority cases.
- It provides a useful baseline but is not a meaningful feature-based model.

8. Algorithm 2 — Memorizer

The lecture defines the Memorizer as follows: if a set of features exists in the training dataset, predict its corresponding label; otherwise predict a random label. Because every training example is stored, the Memorizer always obtains zero training error on the examples it has memorized.

8.1 OOP Implementation

```

import random

class MemorizerClassifier:
    def __init__(self):
        self.memory = {}
        self.labels = []

    def fit(self, X, y):
        self.memory = {}
        self.labels = list(set(y))

        for features, label in zip(X, y):
            self.memory[tuple(features)] = label

        return self

    def predict(self, X):
        predictions = []

```

```

    for features in X:
        key = tuple(features)

        if key in self.memory:
            predictions.append(self.memory[key])
        else:
            predictions.append(random.choice(self.labels))

    return predictions

model = MemorizerClassifier()
model.fit(X_train, y_train)

print(model.predict(X_train))

```

For training examples that are stored in memory, the predictions match the memorized labels, giving zero training error when each feature vector has a consistent label.

8.2 Limitations

- It memorizes examples instead of learning a general rule.
- It can have zero training error but poor test error.
- Unseen feature combinations require a fallback rule; the lecture specifies a random label.
- It can require substantial memory when the training dataset is large.
- It does not provide a useful compact explanation of the relationship between features and labels.

9. Algorithm 3 — Decision Stump

A Decision Stump is a one-feature classifier. According to the lecture, it chooses a feature x_d and, for each value of that feature, predicts the most common label among training examples having that value.

For example, if the selected feature is sneezing, the stump may learn: predict '+' when sneezing = Y and predict '-' when sneezing = N. This is the same form as the lecture's example hypothesis.

9.1 OOP Implementation

```

from collections import defaultdict, Counter

class DecisionStumpClassifier:
    def __init__(self, feature_index=0):
        self.feature_index = feature_index
        self.rules = {}
        self.default_label = None

    def fit(self, X, y):
        groups = defaultdict(list)

        # Group labels by the selected feature value
        for features, label in zip(X, y):
            value = features[self.feature_index]
            groups[value].append(label)

        # Learn majority label for each feature value
        self.rules = {}

```

```

    for value, labels in groups.items():
        self.rules[value] = Counter(labels).most_common(1)[0][0]

    # Fallback label for unseen feature values
    self.default_label = Counter(y).most_common(1)[0][0]

    return self

def predict(self, X):
    predictions = []

    for features in X:
        value = features[self.feature_index]
        predictions.append(
            self.rules.get(value, self.default_label)
        )

    return predictions

# Feature 0 = hives
stump = DecisionStumpClassifier(feature_index=0)
stump.fit(X_train, y_train)

print(stump.rules)
print(stump.predict(X_test))

```

9.2 Choosing a Feature

The lecture explicitly raises the questions: How do we pick which feature to split on? Why stop at one feature? This lab intentionally keeps the stump implementation simple so students can focus on the idea of a one-feature hypothesis. In a stronger learner, the feature would be selected using a criterion such as information gain, Gini impurity, or another split-quality measure.

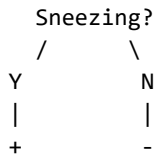
9.3 Limitations

- Uses only one feature, so it cannot represent interactions among multiple features.
- May have nonzero training error.
- A poor feature choice can produce weak predictions.
- It is too simple for many real-world datasets.
- It is less expressive than a multi-level Decision Tree.

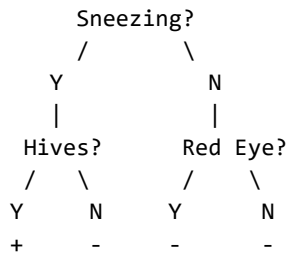
10. Decision Stump → Decision Tree

A Decision Stump can be viewed as the simplest form of a Decision Tree: a tree with only one decision level. The supplied lecture previews Decision Trees immediately after Decision Stump and asks why the learner should stop at one feature. A Decision Tree answers this by allowing additional feature-based decisions at deeper levels.

Conceptual Decision Stump



Conceptual Decision Tree



The stump makes one split. A Decision Tree can make multiple splits and therefore represent more complex decision rules. The lecture also shows that Decision Trees form a hypothesis space for supervised learning.

11. Why Classical Machine Learning Algorithms Are Better Than These Basic Techniques

Majority Vote, Memorizer, and a simple Decision Stump are valuable teaching baselines because their behavior is easy to understand. However, classical ML algorithms are designed to learn useful patterns and generalize to unseen data.

Technique	Uses Features?	Generalization	Main Role
Majority Vote	No	Usually weak	Baseline
Memorizer	Yes, as exact keys	Poor on unseen cases	Demonstrates memorization/overfitting
Decision Stump	One feature	Limited	Simple interpretable rule
Decision Tree	Multiple features	Better potential	Learns hierarchical rules
Other Classical ML	Often multiple features	Designed for generalization	Practical predictive modeling

12. Practical Lab Task

Implement all three algorithms using OOP. Do not use scikit-learn for the core implementation. The purpose is to understand the learning logic before using library implementations.

Task 1 — Prepare the Dataset

```
X_train = [
    ["Y", "N", "N", "N"],
    ["N", "Y", "N", "N"],
    ["Y", "Y", "N", "N"],
    ["Y", "N", "Y", "Y"],
    ["N", "Y", "Y", "N"]
]

y_train = ["-", "-", "+", "-", "+"]

X_test = [
    ["Y", "Y", "Y", "N"],
    ["N", "N", "N", "N"],
    ["Y", "N", "N", "Y"]
]

y_test = ["+", "-", "-"]
```

Task 2 — Implement Majority Vote

Create a class named MajorityVoteClassifier with fit() and predict() methods. Calculate its training and test error.

Task 3 — Implement Memorizer

Create MemorizerClassifier. Verify that it achieves zero training error. Then create unseen test examples and observe its behavior.

Task 4 — Implement Decision Stump

Create DecisionStumpClassifier. Test at least two different feature_index values and compare their training errors.

Task 5 — Compare the Algorithms

```
majority = MajorityVoteClassifier()
majority.fit(X_train, y_train)

memorizer = MemorizerClassifier()
memorizer.fit(X_train, y_train)

stump = DecisionStumpClassifier(feature_index=2)
stump.fit(X_train, y_train)

models = {
    "Majority Vote": majority,
    "Memorizer": memorizer,
    "Decision Stump": stump
}

for name, model in models.items():
    train_pred = model.predict(X_train)
```

```

test_pred = model.predict(X_test)

train_error = classification_error(y_train, train_pred)
test_error = classification_error(y_test, test_pred)

print(name)
print("Training error:", train_error)
print("Test error:", test_error)
print()

```

15. Class Task

Build an OOP-based classifier framework with a common parent class called Classifier. Derive MajorityVoteClassifier, MemorizerClassifier, and DecisionStumpClassifier from it. Add common methods fit(), predict(), and score(). Then compare all models on a larger binary-classification dataset.

```

class BaseClassifier:
    def fit(self, X, y):
        raise NotImplementedError

    def predict(self, X):
        raise NotImplementedError

    def score(self, X, y):
        predictions = self.predict(X)
        correct = sum(
            actual == predicted
            for actual, predicted in zip(y, predictions)
        )
        return correct / len(y)

```

Note: should extend this framework rather than copy-pasting common functionality.

18. Final Kaggle Challenge — Titanic: Machine Learning from Disaster

In this final challenge, students will apply the concepts learned in this laboratory to a real Kaggle classification competition: Titanic — Machine Learning from Disaster. Kaggle describes this as a Getting Started competition designed for learners who are new to machine learning. The task is to predict whether a passenger survived the Titanic disaster. Kaggle provides labeled training data and an unlabeled test set, and students submit predictions to receive an accuracy score and leaderboard position.

Competition: Titanic — Machine Learning from Disaster

Kaggle competition page: <https://www.kaggle.com/competitions/titanic>

The competition provides a natural application of the concepts introduced in this laboratory. The target variable is Survived, where 0 means the passenger did not survive and 1 means the passenger survived. The training data contains 891 labeled passengers, while the test data contains 418 passengers whose survival labels are hidden. Kaggle evaluates the submitted predictions using classification accuracy.

Lab Concept	Titanic Example	Activity
Supervised Learning	Predict Survived from passenger features	Define X and y

Classification	Survived = 0 or 1	Identify the target
Hypothesis $h(x)$	A rule/model predicting survival	Build classifiers
Hypothesis Space H	Possible classification rules/models	Compare simple and classical models
Majority Vote	Always predict the most common class	Build baseline
Memorizer	Remember exact training feature combinations	Observe memorization
Decision Stump	Use one passenger feature	Test Sex, Pclass, Age, etc.
Decision Tree	Use multiple sequential feature decisions	Build a tree model
Generalization	Predict hidden test labels	Submit to Kaggle
Evaluation	Accuracy	Compare local and Kaggle scores

18.3 Step 1 — Load and Inspect the Data

```
import pandas as pd

train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")

print(train.head())
print(train.shape)
print(test.shape)
print(train.info())
print(train["Survived"].value_counts())
```

18.4 Step 2 — Define X and y

For this exercise, students should select a small set of understandable features first. This keeps the focus on the algorithms rather than advanced feature engineering.

```
features = ["Pclass", "Sex", "Age", "Fare"]

X = train[features].copy()
y = train["Survived"].copy()

X_test = test[features].copy()
```

Note: Age and other columns may contain missing values and Sex is categorical. Students should preprocess these values before passing the data to algorithms that require numeric inputs.

18.5 Step 3 — Establish a Majority-Vote Baseline

Use the Majority Vote class implemented earlier in this manual. The purpose is to establish a simple baseline before using feature-based models.

```
majority = MajorityVoteClassifier()
majority.fit(X_train, y_train)

train_predictions = majority.predict(X_train)

print("Training accuracy:",
      1 - classification_error(y_train, train_predictions))
```

18.6 Step 4 — Test the Memorizer

Use the MemorizerClassifier implemented earlier. Compare its training performance with its performance on unseen examples. Discuss why memorizing training examples does not necessarily produce good generalization.

```
memorizer = MemorizerClassifier()
memorizer.fit(X_train, y_train)

train_pred = memorizer.predict(X_train)
print("Memorizer training accuracy:",
      1 - classification_error(y_train, train_pred))
```

18.7 Step 5 — Build Decision Stumps

Create a Decision Stump using one feature at a time. Compare the results. This demonstrates how the choice of a single feature affects the hypothesis.

```
for feature_index, feature_name in enumerate(features):
    stump = DecisionStumpClassifier(feature_index=feature_index)
    stump.fit(X_train, y_train)

    predictions = stump.predict(X_train)
    accuracy = 1 - classification_error(y_train, predictions)

    print(feature_name, "accuracy:", accuracy)
```

18.8 Step 6 — Connect the Stump to a Decision Tree

A Decision Stump makes one feature-based decision. A Decision Tree extends this idea by allowing additional splits. Students should now use a classical Decision Tree implementation and compare it with their manually implemented stump.

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

tree.fit(X_train, y_train)
predictions = tree.predict(X_valid)

print("Validation accuracy:",
      (predictions == y_valid).mean())
```

18.9 Step 7 — Generate Kaggle Predictions

After selecting a final model, train it using the available labeled training data and predict the hidden test passengers.

```
final_model.fit(X, y)

test_predictions = final_model.predict(X_test)
```

18.10 Step 8 — Create the Submission File

Kaggle requires exactly two columns: PassengerId and Survived. The official competition requires 418 test predictions plus a header row. Survived must contain binary predictions: 0 or 1.

```
submission = pd.DataFrame({
    "PassengerId": test["PassengerId"],
    "Survived": test_predictions
})

submission.to_csv("submission.csv", index=False)

print(submission.head())
print(submission.shape)
```

Expected structure:

```
PassengerId, Survived
892, 0
893, 1
894, 0
895, 0
...
```

18.11 Step 9 — Submit to Kaggle

4. Join the Titanic competition on Kaggle and accept the competition rules.
5. Open the Submit Predictions section.
6. Upload submission.csv.
7. Check that Kaggle accepts the file.
8. Record the returned accuracy score.
9. Record the leaderboard rank shown for the submission.

Official competition: <https://www.kaggle.com/competitions/titanic>

18.12 Required Comparison

Complete the following table in your lab report:

Model	Training Accuracy	Validation Accuracy	Kaggle Score
Majority Vote	_____	_____	_____
Memorizer	_____	_____	_____
Decision Stump (best feature)	_____	_____	_____
Decision Tree	_____	_____	_____

18.14 Kaggle Submission Score Sheet

Student Name: _____
 Roll Number: _____
 Kaggle Username: _____

Majority Vote Accuracy: _____
 Memorizer Training Accuracy: _____
 Best Decision Stump Accuracy: _____
 Decision Tree Validation Accuracy: _____

Final Kaggle Score: _____
 Kaggle Rank: _____

Best Model: _____
 Why did you select it?
